

AUDITORÍA FINAL · PECAS + FIXA + ESTETIK + AUDITA

wallet-control

Revisión exhaustiva de código y UI antes de marcar la app como pública y arrancar el build de pruebas con amigos.

FECHA

28 de julio de
2026

VERSIÓN

APP
1.0.0

DISPOSITIVO

PROBADO
Tablet AGS6-W09 ·
Android 10

ALCANCE

Código completo + flujos
en vivo

78%

Lista para la fase de pruebas con amigos — con 3 arreglos antes de repartirla

El día a día (registrar gastos, tarjetas, metas, Finando IA) funciona sólido y se ve pulido. Los problemas que quedan son reales pero acotados: la función nueva de "corte de tarjeta de crédito" no está lista para uso real todavía, y hay un riesgo concreto de que los datos de dos amigos se mezclen si prueban en el mismo celular.

Qué se hizo en esta auditoría: se revisó todo el código fuente por área funcional (Resumen, Tarjetas/corte de crédito, Onboarding/Categorías/Perfil, capa de datos, Finando IA/Historial), y por separado se navegó la app en vivo sobre la tablet conectada, probando de verdad los flujos — incluyendo crear una tarjeta de crédito de prueba, activar su día de corte, registrar un gasto y borrarla, para ver con los propios ojos cómo se comporta la función más nueva de la app.

Lo bueno primero: `npm test --noEmit` sigue en 0 errores, no se encontró ningún `console.log` suelto, la llave de Anthropic nunca se expone al cliente, y la app no se cayó ni una sola vez en toda la sesión de pruebas en vivo. El sistema de colores de los diagramas (arreglado hoy) ya no repite tonos. El flujo completo de registrar un gasto con tarjeta de crédito, verlo reflejado en Balance General, editar la tarjeta y borrarla — se ve limpio y consistente visualmente en cada paso.

Lo que sí hay que atender: el "corte de tarjeta de crédito" (la función más nueva y más compleja que se construyó esta sesión) tiene un bug de fondo que duplica la deuda apenas se activa — se reprodujo en vivo y quedó documentado con capturas. También se confirmó que cambiar de cuenta anónima a cuenta real no limpia los datos de prueba anteriores, lo cual importa porque varios amigos van a probar la app, posiblemente en el mismo teléfono en algún momento.

LOS 3 QUE ARREGLARÍA ANTES DE REPARTIR EL BUILD

1. DOBLE CONTEO AL ACTIVAR EL CORTE DE TARJETA

— es la función más nueva y la que más se promocionó; si un amigo la prueba y ve que la deuda se duplica sola, es el peor primer impresión posible.

2. FUGA DE DATOS ENTRE CUENTA ANÓNIMA Y CUENTA REAL

— con varios amigos probando, es razonable que alguno empiece sin cuenta y luego se registre en el mismo teléfono.

3. BARRA DE NAVEGACIÓN VISIBLE DETRÁS DEL MODAL "REGISTRAR" (ANDROID)

— ya van 3 intentos; es lo primero que cualquiera nota, aunque no rompe nada funcionalmente.

2 cosas que NO son bugs — ya investigadas a fondo, no hace falta reabrir las si aparecen en pruebas futuras:

- El toggle "Confirm email" de Supabase sigue apagado a propósito (beta cerrada) — pendiente operativo, no bug de código.

- El corte de texto en la leyenda de los diagramas de torta, visto *solo en esta tablet específica* — confirmado como límite de renderizado GPU/Skia de ese hardware puntual. Si aparece en el celular de algún amigo durante las pruebas, ahí sí sería un patrón nuevo a investigar.

Bugs — Prioridad alta

1. El corte de tarjeta de crédito duplica la deuda apenas se activa

ALTA

PROBADO EN VIVO

DÓNDE: `components/CardFormModal.tsx:173`

Al activar "Día de corte" por primera vez, la app congela lo ya gastado como la primera deuda del corte — pero lo hace sumando **el total histórico gastado en la tarjeta** (`getCardTotalSpent`) sin filtrar por fecha, en vez de solo lo gastado hasta el día del corte. Como además los gastos originales nunca se eliminan ni se marcan (ver hallazgo #4), el mismo gasto queda contando dos veces: una como "Gastado" normal de la tarjeta, y otra como "Saldo pendiente del corte".

REPRODUCIDO EN VIVO: tarjeta de prueba con \$300.000 gastados → se activó el día de corte → Pasivos en Balance General saltó de \$300.000 a \$600.000 sin registrar ningún gasto nuevo. El panel de la tarjeta mostró simultáneamente "Gastado: \$300.000" y "Saldo pendiente del corte: \$300.000" para el mismo gasto único.

IMPACTO: el patrimonio neto y el diagrama "Billetera general" quedan mal calculados apenas alguien activa el corte en cualquier tarjeta que ya tenía gastos — que es el caso normal de uso (nadie activa el corte en una tarjeta recién creada y vacía).

2. Una tarjeta de crédito nueva creada con "Día de corte" ya puesto nunca activa el sistema de corte

ALTA

PROBADO EN VIVO

DÓNDE: `components/CardFormModal.tsx:171`

La condición `activatingCutoff = isCreditType && card && !card.cutoffDay && !!form.cutoffDay` exige que `card` exista (o sea, que se esté editando una tarjeta ya guardada). Si el día de corte se pone al crear la tarjeta por primera vez, esta condición nunca se cumple y `lastCutoffAt` queda sin definir para siempre — la tarjeta se comporta como si no tuviera corte configurado, aunque el campo esté guardado.

DETALLE EXTRA CONFIRMADO EN VIVO: una vez que `cutoffDay` queda guardado (aunque nunca se haya activado nada), `!card.cutoffDay` es permanentemente falso — ni siquiera volviendo a guardar con el mismo número se activa el corte. La única forma de activarlo es: editar la tarjeta, borrar el día de corte, guardar, volver a editar, poner el día de corte de nuevo, guardar.

IMPACTO: cualquier amigo que cree una tarjeta de crédito y le ponga el día de corte desde el inicio (el camino más natural) nunca verá la función funcionar, sin ningún aviso de que algo falló.

3. "Billetera general" mezcla deuda de tarjetas de crédito con saldos reales, mostrados igual (en verde)

ALTA

PROBADO EN VIVO

DÓNDE: `app/(tabs)/resumen.tsx:557-593`

El diagrama principal de Resumen ("Billetera general") junta `activoItems` (saldos reales: efectivo, débito) con `pasivoItems` (deuda de tarjetas de crédito) en una sola lista, sin distinguirlos visualmente — misma leyenda, mismo color de texto (verde/positivo).

REPRODUCIDO EN VIVO: con la tarjeta de prueba mostrando el bug #1 activo, la leyenda mostró "  Visa Test — \$600.000" en verde, en la misma lista y con el mismo estilo que "Efectivo — \$3.500.000" — dando a entender que esos \$600.000 son dinero disponible, cuando en realidad es deuda (y encima mal calculada por el bug #1).

IMPACTO: confunde el propósito central de la app (saber cuánto dinero tienes realmente) justo en el diagrama que el usuario ve primero al abrir Resumen.

4. Los gastos originales nunca se eliminan ni se marcan al pasar a "saldo del corte"

ALTA

CONFIRMADO EN
CÓDIGO

DÓNDE: `lib/creditCutoff.ts` (`applyPendingCutoffs`) y la activación en `CardFormModal.tsx`

Cuando un corte se procesa (al abrir la app tras la fecha de corte, o al activarlo por primera vez), su monto se suma a `statementBalance` — pero el `Expense` original que generó ese monto sigue existiendo tal cual en el almacenamiento. Como `getCardTotalSpent` sigue sumando esos mismos gastos, la deuda queda contada dos veces de forma permanente, no solo en el momento de la activación (que es el hallazgo #1) sino en cada corte sucesivo que pase de ahí en adelante.

IMPACTO: es la causa raíz común de los hallazgos #1 y #3 — arreglar esto (marcar o excluir los gastos ya "cortados" de `getCardTotalSpent`) resuelve ambos de una vez.

5. Migrar de cuenta anónima a cuenta real no borra los datos de prueba anónimos

ALTA

CONFIRMADO EN
CÓDIGO

DÓNDE: `app/onboarding.tsx` — el flujo de upgrade llama `migrateNamespaceData` pero nunca `wipeNamespaceData('anon')` después.

Si dos personas distintas usan el mismo teléfono/tablet — una empieza en modo anónimo, decide crear cuenta real, y luego otra persona abre la app en modo anónimo en el mismo dispositivo — puede terminar viendo datos que no le pertenecen, porque el espacio de nombres `'anon'` nunca se limpia.

POR QUÉ ES ALTA AHORA: la app está por entrar en una fase de pruebas con varios amigos — es razonable que alguno pruebe primero sin cuenta y decida registrarse después, en su propio teléfono compartido con otra persona de la casa.

Bugs — Prioridad alta (continuación)

6. No hay selector de tarjeta al agregar un gasto desde una categoría

ALTA

CONFIRMADO EN CÓDIGO

DÓNDE: `components/CategoryDetailModal.tsx`

Existe el estado `cardId` pero ninguna parte de la interfaz lo asigna. Un gasto agregado desde el detalle de una categoría nunca debita saldo de ninguna cuenta — el dinero "desaparece" del total de la categoría pero no se refleja en Tarjetas ni en Resumen.

7. Race condition real al borrar una categoría

ALTA

CONFIRMADO EN CÓDIGO

DÓNDE: `lib/storage.ts` — `deleteCategory()`

Escribe en la llave `K_RECURRING_DEFS(ns)` mientras todavía tiene tomado el candado (`withKeyLock`) de `K_CATEGORIES(ns)` , en vez de usar su propio candado. Si dos operaciones sobre gastos recurrentes ocurren casi al mismo tiempo que se borra una categoría, hay una ventana real de condición de carrera. También `syncCardBalanceSnapshot()` , `saveUserProfile` , `deleteUserProfile` , `migrateNamespaceData` y `wipeNamespaceData` no usan el mutex en absoluto.

Bugs — Prioridad media

8. Historial no muestra el emoji de la categoría en "Top categorías" — regresión

MEDIA

PROBADO
EN VIVO

DÓNDE: `app/(tabs)/history.tsx:339`

Antes en esta misma sesión se arregló el bug reportado por el usuario ("cambia ese emoji de los tres puntos por el de la categoría") en el selector de categorías del modal "Registrar" y en Búsqueda. Pero el bloque "Top categorías" dentro del resumen mensual de Historial quedó fuera del arreglo: usa `cat?.icon` directamente (el ícono genérico `Ionicons`, que por defecto es `ellipsis-horizontal`) sin revisar primero `cat?.emoji`, a diferencia de `busqueda.tsx` y `QuickEntryModal.tsx`, que sí lo hacen.

REPRODUCIDO EN VIVO: la categoría "Muchas" (emoji 🎵 elegido por el usuario) se ve correctamente en todos lados excepto en "Top categorías" de Historial, donde muestra "..." en su lugar.

9. Barra de navegación visible detrás del modal "Registrar" en Android — persiste tras el tercer intento de arreglo

MEDIA

PROBADO
EN VIVO

DÓNDE: `components/BottomSheet.tsx` + `app/(tabs)/resumen.tsx`

Se intentó arreglar tres veces esta sesión: (1) ocultar la tab bar vía `navigation.getParent()?.setOptions`, (2) reforzar ese ocultamiento con listeners de teclado, (3) resolver el `maxHeight` porcentual del `BottomSheet` a mano contra `useWindowDimensions` en vez de dejar que Yoga lo calcule contra el modal padre. Se volvió a reproducir en vivo en esta auditoría: al cerrar el teclado dentro del modal "Registrar", el texto "Resumen Tarjetas Finando Historial Perfil" queda visible tenuemente en el borde inferior de la pantalla.

NOTA: no rompe funcionalidad (se pudo registrar el gasto de prueba sin problema), pero es muy visible y ya se intentó tres veces sin éxito — podría valer la pena un enfoque distinto (por ejemplo, revisar si el propio `Modal` de React Native con `statusBarTranslucent` está interactuando mal con `windowSoftInputMode="adjustResize"` en vez de seguir ajustando el cálculo de altura).

10. Inconsistencia entre qué tarjetas se ofrecen para pagar y cuáles realmente alcanzan

MEDIA

CONFIRMADO EN
CÓDIGO

DÓNDE: `lib/recurringPayments.ts` — `getPayableCards` vs. `getCardAvailable`

`getPayableCards` (qué tarjetas se muestran como opción para pagar) filtra tarjetas de crédito solo por `limit - gastado > 0`, sin restar `statementBalance`. En cambio `getCardAvailable` (el que calcula el aviso "supera lo disponible") sí lo resta. Una tarjeta con saldo de corte pendiente que consume todo el cupo real podría seguir apareciendo como opción de pago válida.

11. Cambiar el correo en "Modificar tu cuenta" no sincroniza la tabla de perfiles

MEDIA

CONFIRMADO EN
CÓDIGO

DÓNDE: `app/cuenta.tsx`

Cambiar el correo llama `supabase.auth.updateUser({email})` pero nunca actualiza la columna `profiles.email`. El login por apodo (`get_email_by_nickname`) sigue leyendo el correo viejo y deja de funcionar tras el cambio.

12. Finando IA — nombres de tarjeta por alias no se reconocen

MEDIA

CONFIRMADO EN CÓDIGO

DÓNDE: `lib/expenseParser.ts`

La búsqueda de tarjeta por nombre es exacta — "bbva" no encuentra "BBVA Débito". El gasto se guarda igual, pero sin `cardId`, y como el prompt de Claude trae `askForCard:false`, nunca se le vuelve a preguntar al usuario. Sin validación de `monto>0` ni nombre no vacío antes de guardar tampoco.

13. Wizard de onboarding sin manejo de errores

MEDIA

CONFIRMADO EN CÓDIGO

Los handlers de cada paso (ej. `handlePeriodicityContinue`) no tienen bloques `catch` — si algo falla a mitad del onboarding, no hay ningún aviso visual, el usuario solo ve que "no pasa nada".

14. Contraseñas débiles aceptadas

MEDIA

CONFIRMADO EN CÓDIGO

El único requisito es un mínimo de 6 caracteres, tanto en registro como en cambio de contraseña — vale la pena subir la barra antes de que la app maneje datos financieros reales de varias personas.

15. Al borrar una tarjeta no se cancela su notificación de corte

MEDIA

CONFIRMADO EN CÓDIGO

DÓNDE: `app/(tabs)/tarjetas.tsx`

Al borrar una tarjeta de crédito solo se cancela `notificationId` (la de deuda), pero no `statementNotificationId` (la del recordatorio de pago del corte) — queda una notificación programada para una tarjeta que ya no existe.

16. Pluralización incorrecta en Historial

MEDIA

PROBADO EN VIVO

Con un solo gasto o ingreso registrado en el mes, la tarjeta de Historial muestra "1 gastos" / "1 ingresos" en vez de "1 gasto" / "1 ingreso".

Para eliminar

scripts/generate-icons.js

BAJA

Script de un solo uso, ya corrido, con `sharp` ya desinstalado del proyecto — código muerto reconocido.

assets/images/billetera.png (14 KB)

BAJA

Parece un asset huérfano de una versión anterior — no está referenciado en `app.json`. Confirmar que ningún componente lo importe directamente antes de borrarlo.

Mejoras recomendadas

Falta ícono propio de notificación en app.json

MEDIA

El plugin `expo-notifications` está declarado como string plano, sin `icon` / `color` — Android usa el ícono genérico del sistema en la barra de estado en vez de la marca de la app. Se cubre con bocetos concretos en el PDF de íconos aparte.

Sin suite de pruebas automatizadas

BAJA

No hay Jest ni ningún archivo `*.test.ts` en el proyecto. No es un bug, pero varios de los hallazgos de esta auditoría (el cálculo del corte de tarjeta en particular) son exactamente el tipo de lógica que un puñado de pruebas unitarias detectaría antes de que un usuario real la vea.

Vulnerabilidades reportadas por npm audit

BAJA

34 vulnerabilidades (22 altas, 12 moderadas) reportadas por `npm audit` — las 34 vienen de la cadena de herramientas de build de Expo (`@expo/config-plugins`, `xcode`, `uuid` transitivos), no de código que corra dentro de la app instalada. No urgente, pero vale revisar tras la próxima actualización de Expo SDK.

Comando de skill /estetik no registrado

BAJA

Nota aparte, fuera del código de la app: `SKILLS/ESTETIK.md` y `SKILLS/AUDITA.md` no estaban copiados a `.claude/commands/` pese a que CLAUDE.md decía que sí — ya corregido durante esta sesión. No afecta a la app en sí.

Qué ya funciona bien

- `npx tsc --noEmit` — 0 errores, confirmado al cierre de esta auditoría.
- Ningún `console.log` suelto en el código fuente.
- La llave de Anthropic vive solo en la función `Edge claude-proxy` de Supabase — nunca se expone al cliente.

- El chat de Finando IA bloquea correctamente el modo anónimo.
- La app no se cerró ni mostró un solo error en pantalla durante toda la sesión de pruebas en vivo (crear tarjeta, activar corte, registrar gasto, editar, borrar).
- Los colores de categorías y diagramas ya no se repiten (arreglo de distancia de matiz aplicado hoy) — verificado visualmente en Resumen y Categorías.
- El flujo de registrar un gasto de principio a fin (elegir categoría, elegir tarjeta, monto, guardar, reflejo inmediato en Balance General) funciona sin fricción.
- El panel de acción de una tarjeta (Editar / Eliminar / Abonar al saldo del corte) se ve limpio, con confirmación explícita y clara antes de borrar.
- Perfil, Personalización, Categorías, Finando y Historial se ven visualmente consistentes y responsive en la tablet — tema, tipografía y espaciados uniformes en todas.
- El diálogo de borrado de tarjeta avisa explícitamente si algún gasto va a quedar sin cuenta asignada, en vez de borrarlo silenciosamente.

Checklist pre-lanzamiento (fase de pruebas con amigos)

Actualización — 28 de julio de 2026, tarde: PECAS arregló los 16 hallazgos de código de este informe (los 7 de prioridad alta y los 9 de media) y limpió los 2 ítems de "para eliminar". Los tres primeros — el cluster del corte de tarjeta de crédito (#1, #2, #3, #4), la fuga de datos anónimo→real (#5) y la barra de navegación (#9) — se volvieron a probar en vivo sobre la misma tablet, repitiendo exactamente la reproducción original, y ya no se reproducen. `npx tsc --noEmit` sigue en 0 errores tras todos los cambios. El PDF de íconos y la política de tratamiento de datos siguen pendientes, fuera de este arreglo.

- ☒ **Funcionalidad** — arreglar el doble conteo del corte de tarjeta (#1). Arreglado y reprobado en vivo: activar el corte ya no duplica la deuda.
- ☒ **Funcionalidad** — arreglar la activación del corte al crear tarjeta nueva (#2). Arreglado y reprobado en vivo: una tarjeta creada con día de corte desde el inicio ya activa el sistema correctamente.
- ☒ **Datos** — llamar `wipeNamespaceData( 'anon' )` tras migrar de cuenta anónima a real (#5). Arreglado.
- ☒ **UI** — resolver la barra de navegación detrás del modal "Registrar" en Android (#9). Cuarto intento aplicado (se quitó el `KeyboardAvoidingView` manual en Android, dejando que `adjustResize` nativo maneje el teclado solo) — no se reprodujo en varias pruebas en vivo tras el cambio, pero dado el historial de 3 intentos fallidos previos, vale la pena que el usuario lo confirme también en su celular real antes de darlo por cerrado del todo.
- ☒ **Seguridad** — subido el mínimo de contraseña de 6 a 8 caracteres (#14), en registro y en cambio de contraseña.
- ☒ **Build** — `npx tsc --noEmit` limpio tras todos los arreglos (0 errores).
- ☐ **Assets** — decidir y aplicar el ícono de notificación propio (ver PDF de íconos aparte) antes del build de EAS.
- ☐ **Datos personales** — política de tratamiento de datos redactada y publicada — obligatoria para el formulario "Data safety" de Play Store; en Colombia aplica la Ley 1581 de 2012 (Habeas Data). No se evaluó en esta auditoría de código/UI — pendiente aparte.
- ☐ **Comunicación a los amigos que prueben** — con #1 y #2 ya arreglados, el corte de tarjeta de crédito no necesita el aviso de "experimental" — sigue siendo una función nueva, así que no está de más avisar que es la más reciente y pedir feedback puntual sobre ella.

Resto de hallazgos también arreglados (no estaban en el checklist original)

- **#3** — "Billetera general" ya no mezcla deuda con saldos: ahora muestra el cupo disponible de cada tarjeta de crédito (un activo real), separado del detalle de Pasivos.
- **#4** — causa raíz del doble conteo, resuelta por el mismo arreglo del #1 (nuevo cálculo `getCardCurrentCycleSpent` usado en todos los sitios que antes sumaban gastos sin mirar el corte).
- **#6** — agregado el selector "¿De dónde salió el dinero?" al agregar/editar un gasto desde una categoría.
- **#7** — separados los candados de `deleteCategory()`; agregado mutex a `syncCardBalanceSnapshot`, `saveUserProfile` y `deleteUserProfile`.
- **#8** — "Top categorías" de Historial ya muestra el emoji de la categoría en vez de "...".
- **#10** — `getPayableCards` y `getCardAvailable` ahora calculan el disponible igual, restando también `statementBalance`.
- **#11** — "Modificar tu cuenta" ahora sincroniza `profiles.email` solo. después de que Supabase confirma el cambio de correo.

- **#12** — Finando IA ahora reconoce alias parciales de tarjeta y descarta gastos sin nombre o sin monto real antes de guardarlos.
- **#13** — los pasos del wizard de onboarding que no tenían manejo de errores ahora sí avisan si algo falla.
- **#15** — borrar una tarjeta de crédito ahora también cancela su notificación de corte pendiente.
- **#16** — Historial ya dice "1 gasto" / "1 ingreso" en singular cuando corresponde.
- **Para eliminar** — `scripts/generate-icons.js` y `assets/images/billetera.png` ya se borraron del proyecto.

Auditoría generada combinando revisión de código (criterios AUDITA + FIXA + PECAS) y navegación en vivo sobre dispositivo físico (criterios ESTETIK, vía mobile-mcp) — wallet-control, 28 de julio de 2026. Checklist actualizado la misma tarde tras el arreglo completo de los 16 hallazgos de código.